

Vulnerability Assessment Report

AuSecurity Penetration Test Report

AuSecurity

63 Paddington St
Paddington, NSW 2021

September 17, 2026

Report Version: 1.0

Flinders University
Sturt Road, Bedford Park SA
5042

Table of Contents

List of Figures	4
1 Document Control	5
1.1 Team	5
1.2 List of Changes	5
2 Executive Summary	6
2.1 Overview	6
2.2 Identified Vulnerabilities	7
3 Methodology	8
3.1 Objective	8
3.2 Scope	8
3.3 User Accounts and Permissions	9
4 Findings	10
C1: Default Administrative Credentials (admin:admin)	10
C2: Unauthenticated Remote Code Execution via Unsafe Deserialization (Python pickle)	11
C3: Publicly Accessible osTicket Installer (Unattended Application Setup) ..	12
H1: Sensitive Business/Client Data Exposed via Admin Dashboard	13
H2: Self-Signed TLS Certificate on Main Site	14
M1: Reflected DOM-based Cross-Site Scripting (XSS)	15
M2: Client-Side Open Redirect in Login Flow	16
M3: Directory Listing Enabled on Web Directories	17
M4: WordPress Username Enumeration	18
M5: WordPress XML-RPC Enabled (Pingback SSRF / Multicall Amplification)	19
M6: osTicket Session Cookie Missing HttpOnly Flag	20
I1: Software Version and Technology Disclosure	21
5 Disclaimer	22

A	Appendix	23
A.1	Static Appendix Section	23

List of Figures

Figure 1 - Distribution of identified vulnerabilities	7
---	---

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
---------	-------------	------

2 Executive Summary

2.1 Overview

Executive Summary

AuSecurity is an Australian GRC consulting and penetration-testing firm. On 17 September 2026, an authorized, full-scope security assessment was performed against its public-facing web estate. The goal was to identify weaknesses that an attacker could exploit and to provide practical, prioritized remediation guidance.

Overall Result

The assessment identified **12 findings** in total: **3 critical**, **2 high**, **6 medium**, and **1 informational**. The most serious issue allows a remote attacker to run arbitrary commands on the main web server without logging in. Combined with the other findings, this enables full compromise of the web server and exposure of confidential client information.

What This Means for the Business

- **Server takeover risk.** A flaw in how the website handles login session data lets an attacker execute commands directly on the server. This is the single highest priority to fix.
- **Weak admin protection.** The administrative portal still uses a well-known default password, allowing anyone who knows it to sign in.
- **Confidential data exposure.** Once signed in, the admin dashboard displays client names, contact emails, and financial figures. Because access relies on that default password, this data is effectively public today.
- **Unfinished helpdesk.** The customer support system (osTicket) is live but was never fully installed. Anyone could complete the setup wizard and become its administrator before the business does.
- **Additional weaknesses.** Cross-site scripting, an exposed WordPress interface, a self-signed certificate, and several smaller information leaks further increase risk.

Priority Actions

1. Replace the unsafe session handling with signed, tamper-proof sessions (fixes the critical remote-code-execution issue).
2. Change all administrative passwords and enable multi-factor authentication.
3. Complete and secure the osTicket installation, then remove its setup directory.
4. Fix the cross-site scripting and open-redirect issues.
5. Disable the WordPress XML-RPC interface and username disclosure.
6. Replace the self-signed TLS certificate with a trusted certificate.
7. Apply the remaining hardening steps (directory listings, cookie flags, version banners).

Remediating the critical and high-severity items should be treated as urgent.

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
C1	10.0	Default Administrative Credentials (admin:admin)	10
C2	10.0	Unauthenticated Remote Code Execution via Unsafe Deserialization (Python pickle)	11
C3	9.8	Publicly Accessible osTicket Installer (Unattended Application Setup)	12
H1	8.6	Sensitive Business/Client Data Exposed via Admin Dashboard	13
H2	7.4	Self-Signed TLS Certificate on Main Site	14
M1	6.1	Reflected DOM-based Cross-Site Scripting (XSS)	15
M2	5.4	Client-Side Open Redirect in Login Flow	16
M3	5.3	Directory Listing Enabled on Web Directories	17
M4	5.3	WordPress Username Enumeration	18
M5	5.3	WordPress XML-RPC Enabled (Pingback SSRF / Multicall Amplification)	19
M6	4.3	osTicket Session Cookie Missing HttpOnly Flag	20
I1	0.0	Software Version and Technology Disclosure	21

Vulnerability Overview

In the course of this penetration test **3 Critical**, **2 High**, **6 Medium** and **1 Info** vulnerabilities were identified:

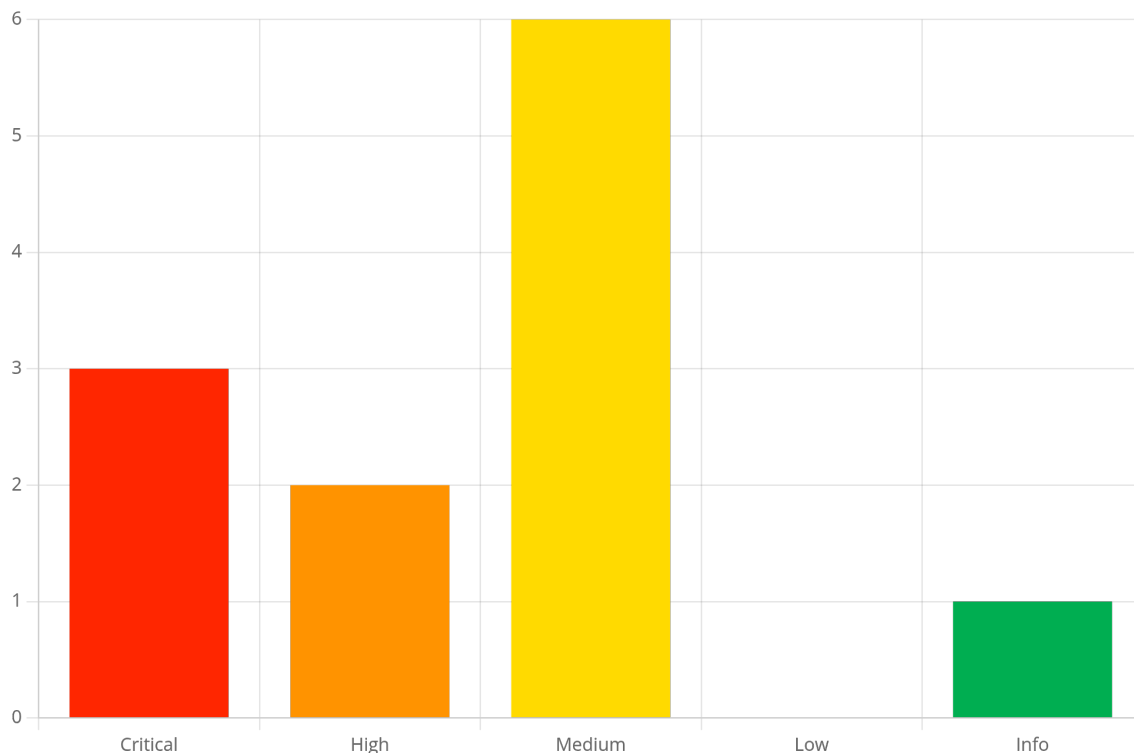


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasible, seamless and cost-affective such approach is.

3.2 Scope

Scope

This engagement was a full-scope, authorized vulnerability assessment of AuSecurity's public-facing assets.

In-Scope Assets

- ausecurity.best (main website, admin login, and API backend) — 51.79.166.148
- www.ausecurity.best and contact.ausecurity.best — 51.79.166.148
- blog.ausecurity.best (WordPress) — 51.79.167.208
- ticket.ausecurity.best (osTicket helpdesk) — 51.79.166.209
- Admin login and dashboard: /api/auth/login and /dashboard

Network services discovered across the in-scope hosts: SSH (22), HTTP (80), HTTPS (443), and HTTP-alternate (8080).

Methodology

- OWASP Web Security Testing Guide for web-application testing.
- Network service enumeration and banner/version discovery.
- Passive OSINT and enumeration, plus non-destructive active testing (port/version discovery and manual web-application testing of the FastAPI-style backend, WordPress, and osTicket).

No brute-force, denial-of-service, or destructive testing was performed.

Limitations

- SSH (port 22) was open on all three hosts but was not brute-forced or exploited; only the version banner was recorded.
- No destructive or denial-of-service testing was performed.
- WordPress did not expose identifiable third-party plugins, so plugin-specific testing was not possible.

- Filesystem-level source review of osTicket was not possible without host access.

3.3 User Accounts and Permissions

No test accounts or credentials were provided. All testing was performed unauthenticated from an external position.

4 Findings

C1: Default Administrative Credentials (admin:admin)	
Score	10.0 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Target	https://ausecurity.best/api/auth/login (51.79.166.148)
References	• CWE-798 (Use of Hard-coded Credentials) • CWE-521 (Weak Password Requirements)

Overview

The administrative login accepts the well-known default credentials `admin / admin`, giving anyone full access to the admin dashboard.

Details

POST `/api/auth/login` with `{"username":"admin","password":"admin"}` returns HTTP 200 and sets the `admin_token` session cookie. With this cookie, GET `/api/auth/me` returns `{"authenticated":true,"user":"admin"}` and the `/dashboard` page is accessible, exposing internal business data (see the "Sensitive Data Exposure" finding). The login endpoint does not distinguish valid vs invalid usernames in error responses, but the default credential itself is trivially guessable and grants administrative access.

Recommendation

Immediately change the admin password to a strong, unique value. Enforce a password policy and add multi-factor authentication for administrative accounts. Remove or rotate any other default/hard-coded credentials.

C2: Unauthenticated Remote Code Execution via Unsafe Deserialization (Python pickle)

Score	10.0 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Target	https://ausecurity.best/ (51.79.166.148) — API backend, `admin_token` cookie
References	• CWE-502 (Deserialization of Untrusted Data) • OWASP A08:2021 Software and Data Integrity Failures

Overview

An attacker can run arbitrary operating-system commands on the AuSecurity web server without logging in. The session cookie is built using Python's unsafe `pickle` serialization and is accepted by the server with no integrity check.

Details

The authentication cookie `admin_token` is a Base64-encoded Python pickle object that wraps a JWT-like string (verified format: `base64(pickle.dumps("|" + jwt))`). The server unpickles this cookie value on every request to the API (e.g. `/api/auth/me`) with no cryptographic signature or MAC protecting the cookie payload. Sending a crafted pickle that reduces to `os.system("<command>")` executes the command server-side before authentication is evaluated.

Proof (timing-based, non-destructive):

- Benign pickle (returns a string): response in ~0.70s, HTTP 401.
- Payload `os.system('sleep 3')`: response in ~3.6s, HTTP 401.
- Payload `os.system('sleep 8')`: response in ~8.8s, HTTP 401.
- Repeated `sleep 3` reproduced at ~3.7s. Delay is proportional to the command's sleep argument, confirming server-side command execution.

This is reachable with no credentials or prior authentication, affecting the application and the underlying host, including any sensitive data and lateral access.

Recommendation

Replace the pickle-based session serializer with a signed, integrity-protected format (e.g. Flask's `itsdangerous` JSON serializer with a strong, random `SECRET_KEY`, or a standard signed JWT). Never deserialize untrusted input with `pickle`. Rotate any secrets/keys and audit the host for signs of compromise.

C3: Publicly Accessible osTicket Installer (Unattended Application Setup)

Score	9.8 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Target	https://ticket.ausecurity.best/ (51.79.166.209), ports 443 and 8080
References	• CWE-489 (Active Debug Code) • CWE-693 (Protection Mechanism Failure)

Overview

The osTicket helpdesk is deployed but never finished installation, and its setup wizard is publicly accessible. Anyone can complete the installation and take over the system as administrator.

Details

Requests to <https://ticket.ausecurity.best/> redirect to `/setup/install.php`, which serves the osTicket v1.18.4 installation wizard (Step 1 prerequisites) with no authentication. The same installer is reachable over plaintext HTTP on port 8080. `/setup/inc/` also returns an open directory listing. Because osTicket is uninstalled, the first person to complete the wizard becomes the administrator; an attacker who reaches it first gains full control of the helpdesk before the business does. The staff panel (`/scp/`) and API (`/api/http.php`, `/api/tickets.php`) all redirect to the installer, confirming the app is not configured.

Recommendation

Complete the installation immediately and restrict access to `/setup/` during deployment. Delete or disable the setup directory after installation (osTicket recommends removing `/setup/`). Ensure the helpdesk is not exposed on plaintext HTTP (8080).

H1: Sensitive Business/Client Data Exposed via Admin Dashboard

Score	8.6 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N
Target	https://ausecurity.best/dashboard (51.79.166.148)
References	• CWE-200 (Exposure of Sensitive Information) • OWASP A01:2021 Broken Access Control

Overview

The admin dashboard discloses client names, contact emails, and commercial metrics. It is reachable using the default admin credentials, so this data is effectively public.

Details

The dashboard (dashboard.html) is embedded with client records (e.g. "James Hargreaves · james@meridianfs.com.au", "Lara Whitfield · lara@shopstream.io", plus four more), together with business metrics (total enquiries 47, active clients 12, revenue YTD \$1.2M) and an activity feed. Because the dashboard is protected only by the default `admin:admin` credential, an unauthenticated attacker can read this confidential client data.

Recommendation

Change the admin credential (see previous finding) and enforce access control so the dashboard is only reachable by authenticated, authorized users. Reassess whether client PII and revenue figures belong in the marketing-hosted application.

H2: Self-Signed TLS Certificate on Main Site

Score	7.4 (High)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N
Target	https://ausecurity.best/ (51.79.166.148)
References	CWE-295 (Improper Certificate Validation)

Overview

The main website serves a self-signed TLS certificate instead of a trusted certificate, undermining transport security and enabling man-in-the-middle attacks.

Details

The certificate for ausecurity.best:443 is self-signed (issuer == subject "CN=ausecurity.best"), valid 2026-09-14 to 2027-09-14, single-name SAN. By contrast, blog.ausecurity.best and ticket.ausecurity.best use valid Let's Encrypt certificates. The self-signed cert causes browsers to show warnings and means clients cannot verify the server's identity, allowing interception of credentials and session tokens.

Recommendation

Replace the self-signed certificate with a trusted certificate (e.g. free Let's Encrypt), consistent with the other subdomains, and automate renewal.

M1: Reflected DOM-based Cross-Site Scripting (XSS)

Score	6.1 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
Target	https://ausecurity.best/consultation-success.html (51.79.166.148)
References	CWE-79 (Improper Neutralization of Input During Web Page Generation)

Overview

The `name` URL parameter is written into the page using `innerHTML` without sanitization, allowing reflected cross-site scripting.

Details

`consultation-success.html` reads the `name` query parameter and assigns it directly: `document.getElementById('user-name').innerHTML = userName;` (line 176). A crafted URL such as `/consultation-success.html?name=<img src=x onerror=alert(1)>` executes attacker-controlled JavaScript in the victim's browser in the context of the AuSecurity origin. This is also reachable through the contact form, whose `full_name` field is reflected into the success redirect (`redirect_url` contains `?name=...`). The `company` parameter uses `textContent` and is safe; only `name` is vulnerable.

Recommendation

Use `textContent` instead of `innerHTML` for the `name` value (as is already done for `company`), or sanitize/encode the value before inserting it into the DOM.

M2: Client-Side Open Redirect in Login Flow

Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	https://ausecurity.best/login (51.79.166.148)
References	• CWE-601 (URL Redirection to Untrusted Site) • OWASP A01:2021

Overview

The login page redirects to an arbitrary URL supplied in the `?redirect=` query parameter after authentication, without validating that the target is same-origin.

Details

In `assets/js/main.js`, both the pre-auth and post-login flows assign `window.location.href = redirect` (from the `redirect` query parameter) with no validation, despite a code comment claiming "only allow same-origin redirects". A crafted link such as `/login?redirect=https://evil.example.com` sends a user to an attacker-controlled site after they log in, enabling phishing/credential capture.

Recommendation

Validate the `redirect` value server-side or client-side to allow only same-origin/relative URLs (e.g. reject values starting with `//` or containing `://`).

M3: Directory Listing Enabled on Web Directories

Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	• https://blog.ausecurity.best/wp-content/uploads/ (51.79.167.208) • https://ticket.ausecurity.best/setup/inc/ (51.79.166.209)
References	CWE-548 (Exposure of Information Through Directory Listing)

Overview

Directory browsing is enabled on the WordPress uploads directory and the osTicket setup includes directory, exposing file names and structure.

Details

<https://blog.ausecurity.best/wp-content/uploads/> returns an "Index of /wp-content/uploads" listing (2026/09 subfolders). <https://ticket.ausecurity.best/setup/inc/> returns an "Index of /setup/inc" listing. These expose internal file names and directory structure that aid further attacks.

Recommendation

Disable directory indexing (e.g. `Options -Indexes` in Apache, or `autoindex off` in nginx) and add a default index document or deny rule.

M4: WordPress Username Enumeration

Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://blog.ausecurity.best/ (51.79.167.208)
References	CWE-200 (Exposure of Sensitive Information)

Overview

The WordPress author archive reveals the administrative username `admin`.

Details

GET `/?author=1` returns a 301 redirect to `/index.php/author/admin/`, disclosing that the user with ID 1 has the login "admin". An attacker can confirm this username to target the admin account (e.g. via the login page or XML-RPC).

Recommendation

Disable author enumeration (e.g. block `?author=` requests or redirect them), and avoid using predictable usernames for privileged accounts.

M5: WordPress XML-RPC Enabled (Pingback SSRF / Multicall Amplification)

Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://blog.ausecurity.best/xmlrpc.php (51.79.167.208)
References	• CWE-749 (Exposed Dangerous Method or Function) • CWE-441 (Unintended Proxy) • OWASP SSRF

Overview

The WordPress blog exposes XML-RPC with 81 methods, including `pingback.ping` (a server-side request forgery vector) and `system.multicall` (request amplification). Custom `demo.*` methods indicate non-standard code is loaded.

Details

`xmlrpc.php` responds with the full method list. `pingback.ping` can be abused to make the server issue outbound HTTP requests to arbitrary targets (SSRF), and `system.multicall` allows many requests to be batched into one (amplification). Two custom methods (`demo.sayHello` returning "Hello!", `demo.addTwoNumbers` returning 3) are also exposed, indicating custom test code has been added to the XML-RPC handler. XML-RPC is unnecessary for modern WordPress and is a common attack surface.

Recommendation

Disable XML-RPC entirely (e.g. block `/xmlrpc.php` at the web server or use a plugin) unless a specific integration requires it. If it must remain, restrict `pingback` and `system.multicall` methods.

M6: osTicket Session Cookie Missing HttpOnly Flag

Score	4.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:N/A:N
Target	https://ticket.ausecurity.best/ (51.79.166.209)
References	CWE-1004 (Sensitive Cookie Without 'HttpOnly' Flag)

Overview

The PHPSESSID cookie set by the osTicket installer lacks the HttpOnly flag, increasing exposure to cookie theft via script injection.

Details

Responses from https://ticket.ausecurity.best/ (and port 8080) set the PHPSESSID cookie without the HttpOnly attribute. If any script execution vulnerability exists in the application, the session identifier could be exfiltrated via JavaScript.

Recommendation

Set `HttpOnly` (and `Secure`, with an appropriate `SameSite` value) on session cookies.

I1: Software Version and Technology Disclosure

Score	0.0 (Info)
Vector string	N/A
Target	ausecurity.best, blog.ausecurity.best, ticket.ausecurity.best (all in-scope IPs)
References	CWE-200 (Exposure of Sensitive Information)

Overview

Multiple services disclose exact software versions, aiding an attacker in selecting exploits.

Details

Observed disclosures: nginx/1.22.1 and nginx/1.25.5 (via Server header and error footers), Apache/2.4.68 (Debian), PHP/8.2.33 (X-Powered-By on osTicket), WordPress 7.1 (generator meta and `?ver=7.1` asset query), osTicket v1.18.4 (installer), and OpenSSH 9.2p1 Debian (banner). Additionally, port 443 advertises HTTP/0.9 via ALPN, which is unusual and unnecessary.

Recommendation

Suppress version banners (`server_tokens off` in nginx, `expose_php = Off` in PHP, `ServerTokens Prod` in Apache) and disable obsolete protocol advertisement.

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerability assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section